ShrinkageTrees: An R Package for Bayesian Tree Ensembles for Survival Analysis and Causal Inference

by Tijn Jacobs, Stéphanie van der Pas, and Wessel van Wieringen

Abstract ShrinkageTrees is an R package for fitting Bayesian tree ensemble models with flexible regularisation mechanisms for survival analysis and causal inference. The package extends classical Bayesian Additive Regression Trees (BART) by separating structural regularisation, induced through tree depth and Dirichlet splitting priors, from shrinkage on the leaf step heights via global–local priors such as the horseshoe. This modular architecture enables principled regularisation in high-dimensional settings. ShrinkageTrees provides unified implementations of BART, DART, Bayesian Causal Forests (BCF), and Horseshoe Forest shrinkage for survival outcomes modelled via accelerated failure time (AFT) formulations with right-censoring and interval-censoring. An efficient Rcpp backend and coherent S3 interface allow users to flexibly combine structural sparsity and parameter shrinkage within a reproducible workflow.

1 Introduction

Bayesian Additive Regression Trees [BART; Chipman et al. (2010)] are a widely adopted non-parametric method for flexible regression and prediction. BART models the response as a sum of m shallow decision trees, each regularised through a Bayesian prior, and can capture complex interactions and non-linearities without requiring the user to specify a parametric form. These properties make BART attractive for the analysis of time-to-event data, where the relationship between covariates and survival is often unknown. Sparapani et al. (2021) extend BART to survival outcomes through both a discrete-time hazard formulation and an accelerated failure time (AFT) approach within the **BART** package. The AFT formulation models $\log T_i$ as a sum of trees plus a noise term, and accommodates right-censored observations through data augmentation in the Gibbs sampler. To date, the **BART** package is, to our knowledge, the only CRAN package that implements Bayesian tree ensembles for survival data. It lacks support for several recent methodological developments that we discuss below.

Beyond prediction, there is growing interest in using tree ensembles for causal inference. Bayesian Causal Forests [BCF; Hahn et al. (2020)] decompose the response surface into a prognostic function $f(\mathbf{x})$, shared across treatment groups, and a treatment effect function $\tau(\mathbf{x})$ that captures heterogeneity in the causal effect. This two-forest structure improves estimation of conditional average treatment effects (CATEs) compared to single-forest approaches. The combination of causal inference and survival analysis arises naturally in clinical trials and observational studies, where the outcome is a time-to-event and treatment effects may vary across patient subgroups. Yet no existing R package provides Bayesian causal forests for survival endpoints.

In high-dimensional settings, where the number of candidate predictors p is large relative to the sample size n, regularisation becomes critical. Standard BART controls complexity through a prior on tree depth and a conjugate normal prior on the leaf step heights $h_{j\ell}$. This does not explicitly distinguish between two different aspects of regularisation: *structural sparsity* — which variables enter the model and how deep trees grow — and *parametric shrinkage* — how strongly individual leaf contributions are pulled towards zero. DART (Linero, 2018) addresses structural sparsity by placing a Dirichlet prior on the splitting probabilities, encouraging the model to concentrate splits on relevant variables. Jacobs et al. (2025) take a complementary approach and apply global–local shrinkage priors, such as the horseshoe (Carvalho et al., 2010), directly to the leaf step heights. The horseshoe prior aggressively shrinks uninformative contributions towards zero while leaving large, well-supported effects essentially unpenalised. We refer to tree ensembles equipped with this prior as *Horseshoe Forests*. In **ShrinkageTrees**, structural and parametric regularisation can be combined freely within a single model, giving the practitioner fine-grained control over the regularisation strategy.

We introduce the **ShrinkageTrees** package, a unified R implementation that brings together these developments. The package supports survival outcomes via AFT models with both right-censoring and interval-censoring; causal inference through BCF with separate prognostic and treatment effect forests, extended to survival endpoints; flexible regularisation by decoupling structural sparsity from parametric shrinkage; and multi-chain MCMC with parallel execution. We implement the core samplers in C++ via **Rcpp** (Eddelbuettel and François, 2011) and provide a coherent S3 interface with `print`, `summary`, `predict`, and `plot` methods. The methodological foundations are developed in Jacobs et al. (2025).

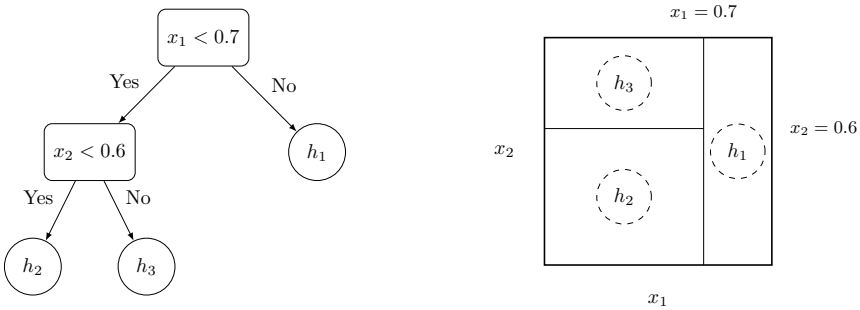

Figure 1: Schematic of a single regression tree. Interior nodes contain binary splitting rules of the form $x_\rho \leq c$; terminal nodes (leaves) contain step heights h_ℓ. An observation traverses the tree from root to leaf, and its prediction is the step height of the leaf it reaches.

The remainder of this paper is organised as follows. Section 2 provides a brief overview of the statistical methodology. Section 3 reviews related R packages and compares their capabilities. Section 4 describes the package architecture and implementation. Section 5 presents an extended worked example. Section 6 concludes with a discussion.

2 Overview of Statistical Methodology

Bayesian Additive Regression Trees

We adapt the Bayesian Additive Regression Trees (BART) model, originally introduced by Chipman et al. (2010) to estimate the conditional mean function $\mathbb{E}[Y_i \mid \mathbf{x}_i]$ in regression settings with continuous uncensored outcomes. The model assumes

$$Y_i = \sum_{j=1}^{m} g(\mathbf{x}_i; \mathcal{T}_j, \mathcal{H}_j) + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma^2),$$

where each $\mathcal{T}_j$ is a binary decision tree that defines recursive partitioning rules over the covariate space, and $\mathcal{H}_j = \{h_{j1}, \ldots, h_{jL_j}\}$ denotes the set of step heights associated with the L_j leaves of tree $\mathcal{T}_j$.

For a given input $\mathbf{x}$, the function $g(\mathbf{x}; \mathcal{T}_j, \mathcal{H}_j)$ maps $\mathbf{x}$ to a leaf node ℓ in $\mathcal{T}_j$ and returns the corresponding value $h_{j\ell}$. Each tree partitions the covariate space into disjoint regions, assigning a constant prediction to each (see Figure 1). The prediction of the j-th tree for individual i is obtained by

$$g(\mathbf{x}_i; \mathcal{T}_j, \mathcal{H}_j) = \sum_{\ell=1}^{L_j} h_{j\ell} \cdot \mathbb{1}(\mathbf{x}_i; \mathcal{T}_j, \ell),$$

where $\mathbb{1}(\mathbf{x}_i; \mathcal{T}_j, \ell) = 1$ if $\mathbf{x}_i$ is assigned to leaf ℓ in tree $\mathcal{T}_j$, and zero otherwise.

A prior is specified over the ensemble $\{(\mathcal{T}_j, \mathcal{H}_j)\}_{j=1}^{m}$ by independently assigning a prior on the tree structure and a conditional prior on the step heights given the tree:

$$p(\mathcal{T}_j, \mathcal{H}_j) = p_{\mathcal{T}}(\mathcal{T}_j)\, p_{\mathcal{H}}(\mathcal{H}_j \mid \mathcal{T}_j) \quad \text{for } j = 1, \ldots, m.$$

These define the two key components of the BART prior. The tree structure $\mathcal{T}$ of each tree is drawn from a prior $p_{\mathcal{T}}(\mathcal{T})$ originally described by Chipman et al. (1998), based on a heterogeneous Galton–Watson branching process (Ročková and Saha, 2020). The process begins at a root node and recursively decides whether to split a node based on its depth d. The probability that a node at depth d is non-terminal is given by

$$p(d) = \alpha\,(1+d)^{-\beta},$$

with hyperparameters $\alpha > 0$ and $\beta > 0$ governing the expected depth and shape of the tree. Given a tree topology, each interior node is assigned a binary decision rule of the form $[x_\rho \leq c]$, where the splitting variable ρ is selected uniformly from the available covariates and the split point c is drawn uniformly from the range of observed values for variable ρ. The splitting rules are constrained to those producing valid partitions, that is, each subset contains a minimal chosen number of observations.

The prior $p_{\mathcal{H}}$ on the step heights $\mathcal{H}$ is defined conditionally on the tree structure $\mathcal{T}$. The step

heights $h_{j\ell}$ are assigned independent Gaussian priors of the form

$$h_{j\ell} \sim \mathcal{N}\left(0,\ \frac{1}{4\sqrt{m}}\right),$$

where the variance is scaled to ensure that the prior places approximately 95% of its mass within the observed range of the response. This prior is conjugate, enabling efficient non-reversible updates using a Gibbs sampler.

The noise is modelled as $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$, with the error variance σ^2 assigned a scaled inverse-chi-squared prior:

$$\sigma^2 \sim \text{Inv-}\chi^2(\nu,\ \psi),$$

where ν denotes the degrees of freedom parameter and ψ the prior scale. We follow Chipman et al. (2010), setting $\nu = 3$ and choosing ψ so that the prior is centred around a rough estimate of σ^2.

Regularisation

We distinguish three regularisation mechanisms, each targeting a different aspect of the model.

Tree structure priors. Standard BART regularisation follows the branching process prior of Chipman et al. (2010): each internal node at depth d splits with probability $\alpha(1+d)^{-\beta}$, where $\alpha \in (0,1)$ and $\beta \geq 0$ discourage deep trees. Under the default values $\alpha = 0.95$ and $\beta = 2$, most trees remain shallow.

Dirichlet splitting priors. DART (Linero, 2018) replaces the uniform distribution over splitting variables with a Dirichlet prior on the splitting probabilities $\mathbf{s} = (s_1, \ldots, s_p)$. By concentrating the prior on sparse probability vectors, DART encourages the ensemble to split on a subset of relevant variables, achieving structural variable selection.

Global–local shrinkage priors on step heights. Rather than regularising the tree structure alone, one can place shrinkage priors directly on the leaf step heights. **ShrinkageTrees** adopts a scale mixture of normals representation,

$$h_\ell \mid \lambda_\ell, \tau, \omega \sim \mathcal{N}(0,\ \omega\, \lambda_\ell^2\, \tau^2),$$

where τ is a global shrinkage parameter shared across all leaves, λ_ℓ is a local scale specific to leaf ℓ, and ω is a fixed scaling constant. The global parameter τ controls the overall degree of regularisation, while the local parameters λ_ℓ allow individual leaves with strong signals to escape shrinkage.

The horseshoe prior (Carvalho et al., 2010) arises as a specific instantiation of this framework by placing independent half-Cauchy priors on both the local and global scales:

$$\lambda_\ell \sim \mathrm{C}^+(0, \alpha_\lambda), \qquad \tau \sim \mathrm{C}^+(0, \alpha_\tau),$$

where $\mathrm{C}^+(0, \alpha)$ denotes the half-Cauchy distribution with scale α. The half-Cauchy has its median at α and possesses heavy tails — its mean and variance do not exist — which allows the local scales to take very large values when the data support a strong effect (Polson and Scott, 2012). This combination aggressively shrinks the contributions of uninformative leaves towards zero while preserving large, data-supported signals. Following Jacobs et al. (2025), we set $\alpha_\tau = \alpha_\lambda = k/\sqrt{m}$. The Dirichlet and horseshoe priors address complementary aspects of regularisation — structural sparsity and parametric shrinkage, respectively — and can be combined within a single model in **ShrinkageTrees**.

Prediction and causal models

The BART formulation above estimates a single function of the covariates — we refer to this as the *prediction model*. For causal inference, we adopt the Bayesian Causal Forest [BCF; Hahn et al. (2020)] decomposition, also known as the τ-learner (Caron et al., 2022). Given a binary treatment indicator $A_i \in \{0, 1\}$ and an estimate of the propensity score $\hat{e}(\mathbf{x}_i) = \hat{P}(A_i = 1 \mid \mathbf{x}_i)$, the *causal model* writes

$$y_i = f(\mathbf{x}_i, \hat{e}(\mathbf{x}_i)) + A_i\, \tau(\mathbf{x}_i) + \varepsilon_i,$$

where f is a prognostic function modelled by a forest of m_f trees and τ is a treatment effect function modelled by a separate forest of m_τ trees. This separation allows the prognostic and treatment effect components to have different regularisation and complexity. The conditional average treatment effect is $\mathrm{CATE}(\mathbf{x}) = \tau(\mathbf{x})$, and the average treatment effect is $\mathrm{ATE} = \mathbb{E}[\tau(\mathbf{X})]$.

More generally, the scalar multiplier on τ in the BCF decomposition need not equal the raw treatment indicator. Writing b_i for the coding applied to subject i, the model becomes $y_i = f(\mathbf{x}_i, \hat{e}(\mathbf{x}_i)) + b_i\, \tau(\mathbf{x}_i) + \varepsilon_i$. Hahn et al. (2020) discuss several choices. *Centered* coding sets $b_i = A_i - 1/2$, which is the original BCF parameterisation and the default in **ShrinkageTrees**. *Binary* coding uses $b_i = A_i$. *Adaptive*

coding sets $b_i = A_i - \hat{e}(\mathbf{x}_i)$, incorporating the estimated propensity score directly into the treatment multiplier; this is the parameterisation adopted in the **bcf** package. Finally, the *invariant* (parameter-expanded) coding of Hahn et al. (2020, Section 5.2) assigns group-specific intercepts $b_0, b_1 \sim \mathcal{N}(0, 1/2)$ and writes $y_i = f(\mathbf{x}_i) + b_{A_i}\,\tilde{\tau}(\mathbf{x}_i) + \varepsilon_i$, where b_0 and b_1 are updated within the Gibbs sampler via conjugate normal updates and the treatment effect is recovered as $\tau(\mathbf{x}_i) = (b_1 - b_0)\,\tilde{\tau}(\mathbf{x}_i)$. This parameterisation is invariant to the labelling of treatment and control. All four options are available in **ShrinkageTrees** via the `treatment_coding` argument.

For survival outcomes, we adopt an accelerated failure time (AFT) formulation. Let T_i denote the event time, C_i the censoring time, $Y_i = \min(T_i, C_i)$ the observed follow-up time, and $\delta_i \in \{0, 1\}$ the event indicator. The prediction model becomes

$$\log T_i = \sum_{j=1}^{m} g(\mathbf{x}_i;\, T_j, H_j) + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma^2).$$

Right-censored observations ($\delta_i = 0$) are handled by imputing $\log T_i$ from a truncated normal distribution within the Gibbs sampler. **ShrinkageTrees** also supports interval-censored outcomes, where the event time is known only to lie in an interval $(l_i, r_i]$; these are imputed analogously. The causal model extends directly to the AFT setting by replacing y_i with $\log T_i$.

Posterior computation

All models are fitted using Markov chain Monte Carlo (MCMC). The outer loop is a Gibbs sampler that cycles through updates of the tree topologies, the leaf step heights, the error variance σ^2, and — where applicable — the shrinkage parameters and censored event times. Trees are updated one at a time via Bayesian backfitting (Chipman et al., 2010): at each sweep, a partial residual is formed by subtracting the contributions of all other trees from the response, and tree j is then updated conditional on the current state of all other trees. The tree topology is modified through Metropolis–Hastings proposals that either grow a terminal node into an internal node with two new leaves, or prune an internal node back to a leaf. Each proposal is accepted or rejected according to the standard Metropolis–Hastings ratio, which balances the likelihood improvement against the prior on tree structure.

Under the standard BART and Dirichlet (DART) priors, where the leaf parameters have a conjugate normal prior, the marginal likelihood of a proposed tree can be computed in closed form and the step heights are integrated out analytically. This yields the birth–death sampler described in Chipman et al. (2010); a detailed algorithmic exposition is given in Kapelner and Bleich (2016). When global–local shrinkage priors such as the horseshoe are placed on the step heights, this conjugacy is lost: the local and global scale parameters λ_ℓ and τ must be sampled explicitly, and the tree proposals become reversible jump MCMC moves that account for the change in the number of scale parameters when a leaf is added or removed. The details of this reversible jump construction, including the proposal distributions and acceptance ratios, are given in Jacobs et al. (2025).

3 Related Packages

Several R packages implement Bayesian tree ensembles. We review five CRAN packages and compare their capabilities to **ShrinkageTrees**. Table 1 provides a summary.

BART (Sparapani et al., 2021) is the most comprehensive general-purpose BART package. It supports continuous, binary, and multinomial outcomes, and is the only other CRAN package to offer Bayesian tree ensembles for survival data. Survival models include a discrete-time hazard formulation (`surv.bart()`), an AFT formulation (`abart()`), competing risks (`crisk.bart()`), and recurrent events (`recur.bart()`). The package also supports Dirichlet splitting priors via `sparse = TRUE`. However, **BART** does not provide global–local shrinkage priors on the leaf step heights, causal forest models, or interval-censored survival outcomes. Multi-chain MCMC is not natively supported; the `mc.*` functions parallelise a single chain rather than running independent chains.

dbarts (Dorie, 2023) provides a fast, lightweight BART sampler for continuous and binary outcomes. Its key feature is an updatable sampler object: predictors and responses can be modified mid-sampling, which makes **dbarts** well suited as a conditional model within a larger Gibbs sampler. The package supports multi-chain MCMC but does not implement survival models, causal forests, or alternative leaf priors.

bcf (Hahn et al., 2020) implements Bayesian Causal Forests with separate prognostic and treatment effect forests for continuous outcomes. It places half-Cauchy and half-Normal hyperpriors on the leaf scales of the two forests, respectively. The package does not support survival endpoints, horseshoe priors, or Dirichlet splitting.

Table 1: Comparison of Bayesian tree ensemble packages in R.

Package	Survival	Interval censoring	Causal (BCF)	Causal + Survival	Horseshoe prior	Dirich
BART	Yes	No	No	No	No	Yes
dbarts	No	No	No	No	No	No
bcf	No	No	Yes	No	No	No
stochtree	No	No	Yes	No	No	No
SoftBart	No	No	No	No	No	No
ShrinkageTrees	Yes	Yes	Yes	Yes	Yes	Yes

Table 2: Fitting functions in **ShrinkageTrees**. The first four are primary functions; the last four are convenience wrappers.

Function	Description	Class
‘ShrinkageTrees()‘	General shrinkage tree ensemble	‘Shrinkage
‘HorseTrees()‘	Horseshoe Forest (convenience wrapper with default horseshoe scale)	‘Shrinkage
‘CausalShrinkageForest()‘	Causal forest with flexible priors	‘Shrinkage
‘CausalHorseForest()‘	Causal Horseshoe Forest	‘Shrinkage
‘SurvivalBART()‘	Standard BART for AFT survival	‘CausalSh
‘SurvivalDART()‘	DART for AFT survival	‘CausalSh
‘SurvivalBCF()‘	BCF for AFT survival	‘CausalSh
‘SurvivalShrinkageBCF()‘	Shrinkage BCF for AFT survival	‘CausalSh

stochtree is a recent package that implements BART, XBART, BCF, and XBCF for continuous, binary, and ordinal outcomes. It supports variance forests for heteroskedastic modelling and provides model serialisation for warm-starting. However, **stochtree** does not implement AFT survival models, Dirichlet splitting priors, or horseshoe shrinkage on the leaf step heights.

SoftBart (Linero and Yang, 2018) replaces the hard axis-aligned splits of standard BART with smooth logistic splitting rules, yielding a posterior mean that is a continuous function of the covariates. This can improve performance when the true regression surface is smooth, but introduces additional computational cost and a fundamentally different regularisation mechanism. **ShrinkageTrees** exclusively uses hard decision splits, as in classical BART; extending the horseshoe and Dirichlet priors to soft splitting rules is an interesting direction for future work but is beyond the scope of this package.

Compared to these packages, **ShrinkageTrees** is the only implementation that combines global–local shrinkage priors on the step heights with Bayesian tree ensembles for survival and causal inference. It is also the only package to support interval-censored survival outcomes within a Bayesian tree ensemble framework and the only package to offer causal forests for survival endpoints.

4 Package Architecture

ShrinkageTrees is available on CRAN and on GitHub at `https://github.com/tijn-jacobs/ShrinkageTrees`. The package is implemented in R with a C++ backend via **Rcpp** (Eddelbuettel and François, 2011). In this section we describe the design and implementation.

Fitting functions and S3 classes

The package provides four primary fitting functions and four convenience wrappers for common model configurations. Table 2 lists all eight functions. The primary functions accept arguments that control the outcome type (`outcome_type`), prior specification (`prior_type`, `local_hp`, `global_hp`), tree structure (`number_of_trees`, `power`, `base`), and MCMC settings (`N_post`, `N_burn`, `n_chains`).

`ShrinkageTrees()` is the most general prediction model function. It accepts a `prior_type` argument that selects the leaf prior: `"horseshoe"` (per-tree global scale), `"horseshoe_fw"` (forest-wide global scale), `"half-cauchy"` (local shrinkage only), `"standard"` (conjugate normal, as in classical BART), or `"dirichlet"` (Dirichlet splitting, as in DART). `HorseTrees()` is a convenience wrapper that fixes the prior to the horseshoe and exposes a single tuning parameter k controlling the half-Cauchy scales via $\alpha_\tau = \alpha_\lambda = k/\sqrt{m}$. The causal functions `CausalShrinkageForest()` and `CausalHorseForest()` mirror

this design for the causal BCF formulation, with independent prior configurations for the prognostic and treatment effect forests.

All prediction model functions return an object of class `"ShrinkageTrees"`; all causal model functions return `"CausalShrinkageForest"`. Both classes provide `print()`, `summary()`, `predict()`, and `plot()` methods. The `summary()` method reports posterior summaries for σ, variable inclusion probabilities, and — for causal models — the ATE with a credible interval. The `predict()` method returns a `ShrinkageTreesPrediction` object containing posterior means and credible intervals. The `plot()` method produces MCMC diagnostics (traceplots and posterior densities for σ), variable importance plots, and — for causal models — ATE and CATE visualisations. Plotting uses **ggplot2**, which is a suggested but not required dependency.

Rcpp backend

We implement the computationally intensive components of the Gibbs sampler in C++ via **Rcpp**. This includes the Metropolis-within-Gibbs updates for tree topology (grow, prune, and change moves), the conditional updates for the leaf step heights, the horseshoe augmentation steps for the local and global scale parameters, the Gibbs update for σ, and the data augmentation for censored observations. The R layer handles input validation, data preprocessing (centering, scaling, log-transforming survival times), hyperparameter calibration, S3 class construction, and post-processing (back-transformation of predictions).

Internally, the C++ code dispatches between two forest implementations depending on the prior. Priors that require reversible jump proposals (`"horseshoe"`, `"horseshoe_fw"`, `"half-cauchy"`) use a `Forest` class with paired grow–prune moves that satisfy detailed balance. The `"standard"` and `"dirichlet"` priors use a separate `StanForest` class with birth–death moves, which is simpler and faster when global scale updates are not needed.

The scale mixture prior on the step heights is encapsulated in a C++ class `ScaleMixture` that follows the strategy pattern. An abstract base class `EtaPrior` defines a common interface — proposing new local parameters, computing the log prior and proposal densities needed for the Metropolis–Hastings ratio, and optionally performing a global update — while concrete subclasses implement the prior-specific sampling logic. The current subclasses are `FixedVariance` (conjugate normal, used by BART and DART), `HalfCauchy` (local shrinkage only), `Horseshoe` (per-tree global–local shrinkage), and `Horseshoe_fw` (forest-wide global shrinkage). A factory function selects the appropriate subclass at run time based on the user's `prior_type` argument. Because the tree update and reversible jump routines interact with the prior only through the `ScaleMixture` interface, adding a new global–local shrinkage prior — for example a regularised horseshoe or a Dirichlet–Laplace prior — requires only implementing a new `EtaPrior` subclass and registering it in the factory, without modifying the sampler logic.

Multi-chain MCMC

All primary fitting functions accept an `n_chains` argument. When `n_chains > 1`, independent chains are dispatched in parallel via `parallel::mclapply()` using `min(n_chains, parallel::detectCores())` cores. After sampling, the posterior draws from all chains are pooled: sample matrices are concatenated row-wise and posterior means are recomputed from the pooled draws. The returned object is a standard `ShrinkageTrees` or `CausalShrinkageForest` instance, so all downstream methods work without modification. Per-chain diagnostics (acceptance ratios, σ traces) are retained for convergence assessment.

Convergence diagnostics via coda

Assessing MCMC convergence is an essential part of any Bayesian workflow, yet most BART implementations leave this task entirely to the user. **ShrinkageTrees** provides a bridge to the **coda** ecosystem (Plummer et al., 2006) through an `as.mcmc.list()` S3 method. This method extracts the posterior draws of σ — split by chain — and wraps each chain in a `coda::mcmc` object, returning a `coda::mcmc.list`. Because **coda** is a suggested rather than required dependency, all functionality remains available when it is not installed. When it is available, `summary()` automatically reports the effective sample size (ESS) and — for multi-chain fits — the Gelman–Rubin $\hat{R}$ statistic. Users who require more detailed diagnostics can pass the `mcmc.list` object directly to any **coda** function, including `geweke.diag()`, `heidel.diag()`, `autocorr.plot()`, and `gelman.plot()`, obtaining a complete convergence assessment without custom post-processing code.

5 Worked Example

We demonstrate the package on the pdac dataset shipped with **ShrinkageTrees**. This dataset is a processed subset of the TCGA pancreatic ductal adenocarcinoma (PAAD) cohort and contains overall survival times, a binary treatment indicator (radiation therapy versus no radiation), clinical covariates, and expression values of approximately 3,000 genes selected by median absolute deviation. The dataset provides a high-dimensional causal survival problem that exercises the main features of the package. We first fit prediction models for survival, then extend the analysis to causal inference using the BCF formulation.

Data and setup

```
library(ShrinkageTrees)
data("pdac")

time      <- pdac$time
status    <- pdac$status
treatment <- pdac$treatment
X         <- as.matrix(pdac[, !(names(pdac) %in% c("time", "status", "treatment"))])

cat("n =", nrow(pdac), "| p =", ncol(X),
    "| censoring rate =", round(1 - mean(status), 2),
    "| treated =", sum(treatment), "/ control =", sum(1 - treatment), "\n")
```

Survival prediction models

We fit two prediction models on the same data: a standard BART model via `SurvivalBART()` and a Horseshoe Forest via `HorseTrees()`. Both use an AFT formulation with right-censoring. The BART model serves as a baseline; the Horseshoe Forest applies global–local shrinkage to the leaf step heights, which is expected to improve regularisation in this high-dimensional setting.

```
set.seed(2025)
fit_bart <- SurvivalBART(
  time           = time,
  status         = status,
  X_train        = X,
  number_of_trees = 200,
  N_post         = 5000,
  N_burn         = 5000,
  n_chains       = 2,
  verbose        = FALSE
)

set.seed(2025)
fit_horse <- HorseTrees(
  y              = time,
  status         = status,
  X_train        = X,
  outcome_type   = "right-censored",
  timescale      = "time",
  number_of_trees = 200,
  k              = 0.1,
  N_post         = 5000,
  N_burn         = 5000,
  n_chains       = 2,
  store_posterior_sample = TRUE,
  verbose        = FALSE
)
```

The `print()` and `summary()` methods provide a concise overview of the fitted model, including the outcome type, MCMC settings, acceptance ratio, and posterior summaries for σ.

```
print(fit_horse)
summary(fit_horse)
```

Convergence diagnostics

We assess convergence using sigma traceplots and posterior densities. With `n_chains = 2`, each chain is plotted separately, allowing visual assessment of mixing and chain agreement.

```
plot(fit_horse, type = "trace")
plot(fit_horse, type = "density")
```

Variable importance

The `plot()` method with `type = "vi"` displays posterior inclusion probabilities for the top predictors. We compare variable importance between the BART and Horseshoe Forest models. The horseshoe prior is expected to produce sparser variable selection by aggressively shrinking the contributions of uninformative predictors.

```
plot(fit_bart,  type = "vi", n_vi = 15)
plot(fit_horse, type = "vi", n_vi = 15)
```

Prediction

The `predict()` method computes posterior predictive distributions on new data, returning posterior means and credible intervals. We illustrate this by predicting on the training data itself.

```
pred <- predict(fit_horse, newdata = X)
print(pred)
summary(pred)
```

Causal survival analysis

We now estimate heterogeneous treatment effects using `CausalHorseForest()`, which fits a causal BCF model with horseshoe shrinkage on both the prognostic and treatment effect forests. We first estimate propensity scores using `HorseTrees()` with a binary outcome, then include the estimated propensity score $\hat{e}(\mathbf{x})$ in the control covariate matrix following Hahn et al. (2020).

```
set.seed(2025)
ps_fit <- HorseTrees(
  y            = treatment,
  X_train      = X,
  outcome_type = "binary",
  k            = 0.1,
  N_post       = 5000,
  N_burn       = 5000,
  verbose      = FALSE
)
propensity <- pnorm(ps_fit$train_predictions)
X_control  <- cbind(propensity, X)

set.seed(2025)
fit_causal <- CausalHorseForest(
  y                        = time,
  status                   = status,
  X_train_control          = X_control,
  X_train_treat            = X,
  treatment_indicator_train = treatment,
  outcome_type             = "right-censored",
  timescale                = "time",
  number_of_trees          = 200,
  k                        = 0.1,
  N_post                   = 5000,
  N_burn                   = 5000,
  n_chains                 = 2,
  store_posterior_sample     = TRUE,
```

```
  verbose                   = FALSE
)
summary(fit_causal)
```

Treatment effect heterogeneity

The `summary()` method for causal models reports the posterior ATE with a 95% credible interval. The `plot()` method provides additional visualisations of treatment effect heterogeneity.

```
# Posterior ATE density
plot(fit_causal, type = "ate")

# Observation-level CATEs with 95% credible intervals
plot(fit_causal, type = "cate")

# Variable importance for both forests
plot(fit_causal, type = "vi", forest = "both")
```

The CATE plot displays patient-level treatment effect estimates sorted by posterior mean, allowing identification of subgroups that benefit most from treatment. The variable importance comparison across the prognostic and treatment effect forests reveals which variables drive the baseline outcome versus which modify the treatment effect.

Comparison of regularisation strategies

To illustrate the modularity of the package, we fit a second causal model using `SurvivalShrinkageBCF()`, which combines Dirichlet splitting priors with half-Cauchy leaf shrinkage. This alternative regularisation strategy encourages structural sparsity in addition to parametric shrinkage.

```
set.seed(2025)
fit_shrinkage_bcf <- SurvivalShrinkageBCF(
  time            = time,
  status          = status,
  X_train         = X,
  treatment       = treatment,
  number_of_trees_control = 200,
  number_of_trees_treat   = 50,
  N_post          = 5000,
  N_burn          = 5000,
  n_chains        = 2,
  store_posterior_sample = TRUE,
  verbose         = FALSE
)
summary(fit_shrinkage_bcf)
```

6 Discussion and Conclusion

We have introduced **ShrinkageTrees**, an R package for Bayesian tree ensembles that combines global–local shrinkage priors with survival analysis and causal inference. The package decouples structural regularisation from parametric shrinkage, allowing practitioners to tailor the regularisation strategy to the problem at hand. To our knowledge, it is the first CRAN package to offer Bayesian causal forests for survival endpoints, horseshoe priors on tree ensemble step heights, and interval-censored survival outcomes within a single framework.

The package has several limitations that suggest directions for future work. Currently, all covariates must be numeric: continuous, binary, and ordered categorical variables (encoded as integers) are handled naturally by the axis-aligned splitting rules, but unordered categorical variables must be dummy-encoded by the user. This can be suboptimal for high-cardinality factors, where native subset splits — as implemented in **bartMachine** (Kapelner and Bleich, 2016) — may be more efficient. The current AFT formulation assumes normally distributed errors; extending the framework to alternative error distributions (e.g. logistic, log-logistic) would broaden its applicability. The Gibbs sampler can be slow to mix in very high-dimensional settings; incorporating warm-starting or accelerated sampling

strategies could improve computational efficiency. Integration with the **coda** package for formal convergence diagnostics (e.g. Gelman–Rubin statistics) is planned.

ShrinkageTrees is available on CRAN and actively maintained on GitHub at https://github.com/tijn-jacobs/ShrinkageTrees. The methodological foundations are developed in Jacobs et al. (2025).

7 Funding

Tijn Jacobs and Stéphanie van der Pas received funding from the European Research Council (ERC) under the European Union's Horizon Europe program under Grant agreement No. 101074082. Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or the European Research Council Executive Agency. Neither the European Union nor the granting authority can be held responsible for them.

Bibliography

A. Caron, G. Baio, and I. Manolopoulou. Estimating individual treatment effects using non-parametric regression models: A review. *Journal of the Royal Statistical Society Series A: Statistics in Society*, 185(3): 1115–1149, 2022. doi: 10.1111/rssa.12889. [p3]

C. M. Carvalho, N. G. Polson, and J. G. Scott. The horseshoe estimator for sparse signals. *Biometrika*, 97(2):465–480, 2010. doi: 10.1093/biomet/asq017. [p1, 3]

H. A. Chipman, E. I. George, and R. E. McCulloch. Bayesian CART model search. *Journal of the American Statistical Association*, 93(443):935–948, 1998. doi: 10.1080/01621459.1998.10473750. [p2]

H. A. Chipman, E. I. George, and R. E. McCulloch. BART: Bayesian additive regression trees. *The Annals of Applied Statistics*, 4(1):266–298, 2010. doi: 10.1214/09-AOAS285. [p1, 2, 3, 4]

V. Dorie. *dbarts: Discrete Bayesian Additive Regression Trees Sampler*, 2023. URL https://CRAN.R-project.org/package=dbarts. R package version 0.9-23. [p4]

D. Eddelbuettel and R. François. Rcpp: Seamless R and C++ integration. *Journal of Statistical Software*, 40(8):1–18, 2011. doi: 10.18637/jss.v040.i08. [p1, 5]

P. R. Hahn, J. S. Murray, and C. M. Carvalho. Bayesian regression tree models for causal inference: Regularization, confounding, and heterogeneous effects. *Bayesian Analysis*, 15(3):965–1056, 2020. doi: 10.1214/19-BA1195. [p1, 3, 4, 8]

T. Jacobs, W. N. van Wieringen, and S. L. van der Pas. Horseshoe forests for high-dimensional causal survival analysis. *arXiv preprint arXiv:2507.22004*, 2025. doi: 10.48550/arXiv.2507.22004. [p1, 3, 4, 10]

A. Kapelner and J. Bleich. bartMachine: Machine learning with Bayesian additive regression trees. *Journal of Statistical Software*, 70(4):1–40, 2016. doi: 10.18637/jss.v070.i04. [p4, 9]

A. R. Linero. Bayesian regression trees for high-dimensional prediction and variable selection. *Journal of the American Statistical Association*, 113(522):626–636, 2018. doi: 10.1080/01621459.2016.1264957. [p1, 3]

A. R. Linero and Y. Yang. Bayesian regression tree ensembles that adapt to smoothness and sparsity. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 80(5):1087–1110, 2018. doi: 10.1111/rssb.12293. [p5]

M. Plummer, N. Best, K. Cowles, and K. Vines. CODA: Convergence diagnosis and output analysis for MCMC. *R News*, 6(1):7–11, 2006. URL https://journal.r-project.org/archive/. [p6]

N. G. Polson and J. G. Scott. On the half-Cauchy prior for a global scale parameter. *Bayesian Analysis*, 7 (4):887–902, 2012. doi: 10.1214/12-BA730. [p3]

V. Ročková and E. Saha. On the theory of BART. *arXiv preprint arXiv:1910.00597*, 2020. [p2]

R. Sparapani, C. Spanbauer, and R. McCulloch. Nonparametric machine learning and efficient computation with Bayesian additive regression trees: The BART R package. *Journal of Statistical Software*, 97(1):1–66, 2021. doi: 10.18637/jss.v097.i01. [p1, 4]

Tijn Jacobs
Vrije Universiteit Amsterdam
Department of Mathematics
Amsterdam, The Netherlands
https://tijn-jacobs.github.io/
ORCiD: 0009-0003-6188-9296
t.jacobs@vu.nl

Stéphanie van der Pas
Vrije Universiteit Amsterdam
Department of Mathematics
Amsterdam, The Netherlands
https://www.stephanievanderpas.nl/
ORCiD: 0000-0002-2448-5378
s.l.vander.pas@vu.nl

Wessel van Wieringen
Vrije Universiteit AmsterdamAmsterdam University Medical Centre
Department of Mathematics, Amsterdam, The Netherlands
Epidemiology and Data Science, Amsterdam, The Netherlands
https://www.math.vu.nl/~wvanwie/
ORCiD: 0000-0002-5100-9123
w.n.vanwieringen@vu.nl